Manent: File-First, Git-Versioned Memory for AI Agents

Specification, evaluation harness, and retrieval ablations on atomic notes

Salvatore Colopi

Independent software engineer, Italy. hello@salvatorecolopi.com. Project page: salvatorecolopi.com/manent. Code: github.com/colopisalvatore/manent (Apache-2.0). Preprint, 4 September 2026.

Abstract. Memory for AI agents is usually either a proprietary vector store that "extracts" memories from conversations (opaque, not diffable, not curatable by hand) or an unstructured pile of notes with no schema and no guarantee of retrieval quality. We present **Manent**, a third path: memory is a plain Markdown vault (Obsidian-compatible), versioned in git, governed by a closed, versioned frontmatter specification with typed edges, and served to any agent through the Model Context Protocol. Every index, graph and embedding is derived and rebuildable from the files, which are the only source of truth. The contribution is threefold. (1) A specification that adds to the note contract an *audience* field whose absence means *private*, and a serving model in which each identity reads the vault through a view built *before* any index exists, so that tools that bypass the ranker cannot bypass the access filter. (2) An evaluation harness with a regression gate and three kinds of query: hand-written close to the note's wording, hand-written *oblique* (the concept without the note's words), and automatically derived from note descriptions. It turned ranking into a measured decision. (3) The measurements themselves, on a real 479-note vault: a tokenization rule was worth 35 points of top-1 accuracy before any semantic method; graph expansion by Personalized PageRank did not improve retrieval on the reference vault; fusing lexical and local dense retrieval by reciprocal rank reached the best top-1 accuracy, with the lexical/dense balance proving corpus-sensitive enough to deserve periodic re-measurement; and splitting notes into passages under plain max-pooling made retrieval *worse*, because it reintroduces the length bias BM25 normalizes away; a length-aware aggregation or a bounded number of passages recovers it, and a contextual prefix on every passage proved indispensable. We also describe a *gap register* that lets a shared brain learn from the questions it could not answer without storing anyone's text, and which manufactures oblique golden-set queries from real use. The paper is best read as an empirical study of file-based agent memory, with a reproducible specification and evaluation harness, rather than as a new memory architecture competing with systems that extract memories from conversations. Limitations are substantial and stated: one corpus, a small author-written golden set, no end-task evaluation, no user study.

Keywords: agent memory, retrieval-augmented generation, hybrid retrieval, reciprocal rank fusion, chunking, evaluation, Model Context Protocol, access control

1. Introduction

Large-language-model agents forget everything at the end of a context window. The systems built to fix this fall into two families. The first extracts "memories" from conversations and stores them as embeddings and rows in a database the operator does not read, diff, correct by hand, or carry elsewhere [1, 3, 4]. The second keeps human-readable notes but offers no contract: no schema, no lint, no measured retrieval, no way to know whether a new note broke anything. Both families leave the same question unanswered once more than one agent depends

on the memory: *who wrote this, who may read it, and how do we know retrieval is not getting worse?*

Manent takes a third path, built on four commitments. **Markdown files are the only source of truth**; search indexes, graphs and embeddings are derived artifacts, rebuildable from the files at any time, and a tool that makes derived state authoritative is non-conforming. **Git is the synchronization and audit backbone**: every memory write is a commit with an author, a time and a message. **A closed, versioned schema** (JSON Schema 2020-12) governs note types, frontmatter fields and typed edges, and is linted in CI so that a malformed note never lands. **MCP is the access layer** [12]: any client that speaks the Model Context Protocol mounts the brain with a URL and no custom SDK.

The design exists to make the measurements meaningful; the measurements are the focus of this paper. Three of the four retrieval improvements we expected to ship (graph expansion, passage chunking, and a "hybrid" ranker with recency and centrality multipliers) measured neutral or negative on a real vault, and the one that measured best, a tokenization rule, is the simplest. Without an evaluation harness we would have shipped the wrong three and credited the wrong one. The progression is easy to read: 35% top-1 accuracy with a naive lexical index, 70% once tokenization is fixed, 80% with a local dense model alone, 90% fusing the two; graph expansion stays at 70%, and chunking with plain max-pooling can make the hardest queries worse. Section 6 reports those ablations in full, including the results that contradicted our expectations.

Contributions. (i) A vault specification for agent memory with typed notes, typed edges, wikilink resolution rules that make a vault read the same in an editor and in an index, an *audience* field whose absence means private, and a status field that ranking consumes (§3). (ii) A serving model in which identity resolves to a view of the vault built before indexing, writes by agents are gated, quarantined and confirmed by the person through the protocol's own multi round-trip primitive, and every call is auditable (§4). (iii) An evaluation harness whose three query kinds measure different things, with a regression gate that has already caught one drift (§5). (iv) Retrieval ablations on a real 479-note vault, with one negative and one cautionary result that generalize to any memory made of atomic, front-loaded notes (§6). (v) A gap register that learns from unanswered questions without storing users' text and turns closed gaps into the golden-set queries hardest to write by hand (§7).

2. Related work

Agent memory systems. MemGPT [1] treats the context window as main memory and pages information in and out of external storage under the model's control. Generative Agents [2] introduced a memory stream scored by recency, importance and relevance; Manent's early "hybrid" ranker borrowed those multipliers and measured them away (§6.3). Mem0 [3] and Zep [4] extract and consolidate memories from conversations into a store, a graph in Zep's case, optimized for the recall needed by a conversational agent. These systems own the representation; Manent does not, by design: its representation is files a person edits in an ordinary editor, and the system's job is to index, serve and measure them.

Retrieval. BM25 [5] remains the lexical baseline and, as §6.1 shows, the tokenizer feeding it matters more than the model above it. Dense retrieval with small multilingual encoders [6] is now cheap enough to run on a CPU; we use it locally, without an API, because the vault's text must not leave the machine. Reciprocal rank fusion [7] combines ranked lists by position rather

than score, which is what allows an inverted index and a cosine similarity to be merged without inventing a scale between them. Contextual retrieval [8] prepends a document-level summary to each chunk before embedding; our chunking sweep (§6.4) reproduces the size of that effect and adds a finding on when chunking itself should be avoided. Retrieval-augmented generation [9] frames the whole pipeline; Personalized PageRank [10] is the graph expansion we measured.

Knowledge management. The vault layout follows the Zettelkasten practice of atomic notes [11] and the conventions of Obsidian, whose wikilink resolution by file name and path our specification reconciles with resolution by canonical name (§3.4).

Protocol and access control. The Model Context Protocol revision 2026-07-28 [12] removed sessions and the initialization handshake and introduced multi round-trip requests, which we use for write approval (§4.3). The practice of enforcing document-level permissions *inside* retrieval rather than after ranking is well established in enterprise retrieval systems; Manent applies it at the earliest possible point, the loading of the vault for a given reader (§4.1).

3. The Manent specification (v0.2)

3.1 Core invariant and note contract

One note is one file is one atomic fact, entity or lesson. YAML frontmatter is validated against a base JSON Schema (draft 2020-12) that requires three fields: `name` (a slug equal to the file name, the wikilink target), `description` (one line of at most 300 characters: the surface a ranker judges relevance on without opening the note), and `type`, from a closed enumeration of eleven types (`feedback`, `reference`, `project`, `wiki-entity`, `wiki-concept`, `raw-source`, `persona`, `handoff`, `retro`, `moc`, `index`). Optional fields carry dates, tags, provenance, confidence, status, audience and author; unknown fields are allowed for forward compatibility. Notes are hand-written, so invalid YAML happens; a conforming tool loads such a note with empty frontmatter and an intact body and reports it, instead of failing the run.

3.2 Typed edges

Four edge kinds form a graph over notes: `wikilink` (from `[[name]]` in the body; an unresolved target is not an error but a note worth writing), and three frontmatter edges: `provenance` (from a synthesis to the raw source it came from), `supersedes` (this note replaces that one), `contradicts` (a flagged conflict, surfaced for a person and never auto-resolved).

3.3 Audience and status

Visibility is a property of the note, declared in its frontmatter as `audience`, a list of slug labels, and not of the folder, because the same note often serves two audiences and a folder forces it into one home. Two labels are reserved. `private` is what a note means when the field is absent or empty: only the owner reads it. This default is the most restrictive reading on purpose, so that a note written without thinking about it cannot become visible by accident. `public` is the one label that may leave the organization. Every other label belongs to the vault, which closes its own set at the lint gate (`--audiences tech,business,product`): an unknown label is reported and, until fixed, makes a note *less* visible, never more, because no reader's scope names it. `status` gains the value `quarantine` for notes written by an agent with no human in the loop; ranking must place `quarantine`, `deprecated` and `archived` notes below an active note of equal relevance (§6.5).

3.4 Wikilink resolution

A vault is written by a person in an editor and read by a machine through an index, and the two do not identify a note the same way: Obsidian resolves `[[foo]]` by file name and `[[dir/foo]]` by path, while a note's identity in the index is its canonical `name`. Where those disagree, as with a note named `moc-ops` living in `moc/ops.md`, a link that works in the editor looks broken to the index. The specification therefore requires a resolver to try, in order: canonical name; path relative to the linking note; path from the vault root; file name alone, when unique or disambiguated by the linking note's own directory. Ambiguity that proximity cannot settle stays unresolved: guessing would wire the graph to the wrong note silently. On the reference vault this rule raised resolved wikilink edges from a state where path-form links were all reported broken to 1,335 of 1,490 resolved, and every remaining unresolved link names a note that does not exist.

3.5 Lint

Twelve rules with three severities: schema, missing or invalid frontmatter and duplicate names are errors; name/filename mismatch, unresolved links, missing body sections for `feedback` notes, misplaced raw sources, personal data, model-directed text and unknown audience labels are warnings that `--strict-content` turns into errors for CI; orphans are informational. Run in strict mode on the brain repository, lint is the gate: a note that fails it never lands in the shared branch, which is the only place the server reads from.

4. Serving a vault to several agents

The server exposes nine tools over MCP: `brain_search` (ranked, returning a `searchId`, §7), `brain_read` and `brain_read_raw`, `brain_neighbors`, `brain_list`, `brain_grep`, `brain_feedback`, and, advertised only when the operator opts in, `brain_write` and `brain_append`. Two protocol eras are served as separate implementations rather than one blended path: the initialization-handshake revisions through the official SDK, and revision 2026-07-28 natively (no handshake, per-request protocol fields, cacheable list results, multi round-trip requests). Tool definitions live in one module and every call passes through one dispatcher, so the eras cannot drift in capability. Era detection keys on the RPC itself and never on a transport header alone, a lesson learned from a dual-era client that sent the newer header with the older handshake and was wrongly rejected. The protocol details in this section are an engineering contribution that will age with the protocol; the access model of §4.1 is the part meant to outlast it.

4.1 Identity resolves to a view, before indexing

A vault read by one person needs no identities: the token is the password. A vault read by a customer-care agent, a coding agent and the owner's own sessions needs to know who is asking. Agents are declared in a JSON file (`name: {token, read: [labels], write: dir}`); the owner's token, an agent's token, and OAuth 2.1 access tokens minted for either all resolve to an *identity* that travels with the call. The identity's *view* of the vault is built from the notes its scope may read (notes, graph and ranker alike) **before any index exists over it**. Search, listing, regex grep, raw reads and neighbourhoods are all computed on the view; edges into hidden notes are dropped, so a neighbourhood cannot even name what its reader may not open. The order is essential: filtering after ranking would leave every tool that bypasses the ranker, grep and raw read among them, unprotected. This is an architectural property, enforced by construction and covered by tests; it is not an audited security guarantee (§8). Views are cached per scope and

invalidated whenever the served state changes; a dense index is sliced by visible note rather than re-embedded.

4.2 Writes: the gate and quarantine

Every write is scanned before anything touches the disk. Text that carries personal data (email addresses, phone numbers, IBANs, card numbers with a valid Luhn checksum, national identifiers) or that reads as an instruction aimed at a model (override phrases in two languages, hidden HTML directives, role hijacks, zero-width and bidirectional control characters, active content) is refused with the reason. The rationale is twofold: a vault lives in git and git history is forever, so the check belongs before storage, not in a cleanup afterwards; and a knowledge base that agents with tools read must not be a place where an outsider's words can become instructions. A write by any identity other than the owner lands in the directory that identity was granted and nowhere else, stamped `status: quarantine`, `author: <agent>` and `audience: [private]` regardless of what the call asked for. A quarantined note is ranked below active ones and visible to the owner only, who promotes it by editing status and audience: a commit with a name on it.

4.3 Approval through the protocol

On revision 2026-07-28 a write does not complete on the first call. It answers `resultType: "input_required"` with an elicitation form showing the proposed note, and completes on the retry that carries the person's confirmation in `inputResponses`. The `requestState` echoed by the client is a fingerprint of what was proposed, so an altered retry is asked again rather than trusted, and the server keeps no state. The agent proposes, the person confirms, in the standard's own primitive. Clients that cannot ask, meaning the handshake eras and clients that do not declare the elicitation capability, fall through: the owner's write goes straight through, an agent's goes to quarantine.

4.4 Audit and hot reload

One JSON line per tool call records the identity, the tool, redacted arguments, result names and latency: enough to reconstruct an incident, not enough to reconstruct a customer. The server watches the vault and re-indexes what changed, coalescing bursts (a `git checkout` that touches hundreds of files costs one reload) and re-embedding only notes whose content hash moved; measured on the test vault, a new file is searchable about 530 ms after the write. A brain that lives in git therefore needs only a post-receive hook that checks out the shared branch.

5. The evaluation harness

Ranking changes are decided by the harness, not by intuition. A retriever is scored against a set of queries, each with the canonical names of the notes a good ranking must surface, and reported as hit@1, hit@3, recall@5, recall@10, MRR and nDCG@10, overall and per query kind. Three kinds answer three different questions:

- **curated**: hand-written, wording close to the note's own: *lexical recall*. Twenty queries on the reference vault (mean 7.0 words; four name more than one acceptable note).
- **oblique**: hand-written, asking for the concept *without* the note's words: *semantic recall*, the hard case. Eight queries (mean 8.9 words). Example: "who pays the two euros on the receipt" targets a note about stamp duty that never says *two euros*.

- **auto**: derived from each note's `description` stripped of stopwords and markup, whose only right answer is that note: a broad regression signal with no labelling cost. 444 queries (mean 8.4 words) on the reference vault; synthetic, so it measures "can retrieval find a note from its own summary", not user phrasing, and is read alongside the curated set, never instead of it.

A saved report acts as a baseline; a later run exits non-zero if any overall metric drops beyond a tolerance of 0.5 points. The server and the harness share one retrieval implementation, so what is measured is what ships, status demotion included since v0.2.

6. Retrieval ablations on a real vault

6.1 Setup

Corpus. The author's working vault: 479 notes at the time of measurement (4 September 2026), written in Italian over fourteen months, of the types listed in Table 1. Notes are atomic and front-loaded, one fact each, stated at the top, with a median body of 3,188 characters (quartiles 1,698 and 6,172; 90th percentile 10,375; a few long transcripts up to 210k). 404 notes carry wikilinks, 1,490 link edges in all. The vault is private and not distributed; the golden set that names its notes is in the repository, and every script takes a vault path so the same measurements run on any conforming vault.

type	notes	type	notes
feedback	145	handoff	28
project	115	persona	20
reference	99	raw-source	13
retro	47	moc / index	10 / 3

Table 1. Reference vault by note type (480 notes when counted; 479 in the measurements below, before one note was added).

Rankers. Lexical: BM25 as implemented by MiniSearch over four fields (name, name split into words, description, body) with field boosts 3/3/2/1. Dense: `Xenova/multilingual-e5-small` [6], 384 dimensions, 8-bit quantized (~120 MB), run locally on CPU through `@huggingface/transformers`, with the E5 "query:" / "passage:" prefixes; each note embedded as one passage made of its title, its description and the first 1,400 characters of its body, vectors cached by content hash. Fused: reciprocal rank fusion [7] of the top-30 of each list with $k = 60$. Hybrid: BM25 candidates re-ranked by Personalized PageRank [10] over wikilinks plus recency and centrality multipliers, kept for the ablation. Hardware: a consumer laptop; embedding 479 passages takes about two seconds once the model is loaded.

6.2 Tokenization was the first big win

The first lexical configuration indexed every term, with prefix and fuzzy matching enabled on all of them. Its curated hit@1 was 35.0%. Dropping Italian and English function words and allowing prefix matching only on terms of at least five characters (fuzzy on six) moved it to 70.0%: 35 points from a tokenizer rule, before any semantic method (Table 2). The mechanism is visible in the misses: with prefix matching on, *di* matches *diritto*, *disposizione*, *documento*, and a long legal note accumulates enough noise to outscore the short note that actually answers. The auto set barely moves (from 96.8% to 97.3%): a query made of a note's own description

words matches it under any tokenizer, which is exactly why that set cannot replace hand-written queries.

lexical configuration	curated hit@1	curated MRR	curated recall@5	oblique MRR	auto hit@1
every term indexed; prefix and fuzzy always on	35.0%	0.557	77.5%	0.000	96.8%
stopwords removed; prefix from 5 chars, fuzzy from 6 chars	**70.0%**	**0.838**	**97.5%**	**0.063**	**97.3%**

Table 2. The tokenization ablation (`scripts/bm25-naive-baseline.mjs`). Same index, same queries, same vault.

6.3 Graph expansion did not pay

Notes with wikilinks form a graph, and expanding lexical results along it looked like an obvious gain. We ran a grid over the graph weight, the hub penalty, the importance boost and the recency half-life (Table 3). No configuration beats the lexical baseline on the curated set, and the multipliers cost points on the auto set. Our interpretation is that Personalized PageRank amplifies a good seed but cannot create one. When the lexical match starts wrong, the walk inherits the error. The hybrid ranker stays available, opt-in, for vaults with a much denser link structure than this one.

configuration	curated hit@1	curated MRR	auto hit@1	auto MRR
BM25 only (lexical baseline)	**70.0%**	**0.838**	**97.3%**	**0.986**
graph .2, hub .4, imp 0, rec off	70.0%	0.838	97.3%	0.986
graph .1, hub .4, imp 0, rec off	70.0%	0.838	97.3%	0.986
graph 0, hub .4, imp 0, rec off	70.0%	0.838	97.3%	0.986
graph 0, hub 1 (no penalty), imp 0, rec off	70.0%	0.838	97.3%	0.986
graph .2, hub .15, imp 0, rec off	70.0%	0.838	97.3%	0.986
graph .3, hub .4, imp 0, rec 180	70.0%	0.838	94.8%	0.971
graph 0, hub .4, imp .15, rec 365	70.0%	0.838	92.3%	0.961
graph .2, hub .4, imp .15, rec 365	70.0%	0.838	92.1%	0.960
graph .5, hub .4, imp .15, rec 365 (the original "hybrid" default)	70.0%	0.838	92.1%	0.960

Table 3. Graph-expansion grid (`scripts/tune-retrieval.mjs`), sorted by curated MRR. "graph" is the weight of the PageRank mass in the final score, "hub" the penalty on high-degree notes, "imp" the importance boost, "rec" the recency half-life in days.

6.4 Lexical and dense fail in opposite directions

The dense ranker alone finds notes BM25 cannot: curated hit@1 rises to 80.0% and oblique recall@5 from 12.5% to 31.3%. It also blurs the exact identifiers (slugs, error codes, entity names) that BM25 nails. Fusing the two lists by rank keeps both strengths without inventing a scale between an inverted index and a cosine similarity. The balance was chosen by sweep on the August snapshot, where equal weights let the lexical list pull correct answers off the top spot (80% curated hit@1) and weighting the dense list twice reached 100%; that is the shipped default. Re-run on the September vault (Table 4), the two weightings tie on the curated set

(90.0%, MRR 0.942), and equal weights are slightly better on the auto set (97.3% vs 96.6%) and on oblique recall@5 (25.0% vs 12.5%, two queries of eight). Heavier dense weights lose curated accuracy. On the oblique set one query is worth 12.5 points of recall@5 and about 0.02 of MRR, so differences of that size (0.181 against 0.192, 12.5% against 25.0%) are exploratory, not findings. The lesson is that the balance is corpus-sensitive and drifts as the vault grows; re-sweeping it is a periodic, single-command task rather than a design decision. The oblique set is too small to rank configurations by, and we do not.

ranker	curated hit@1	curated MRR	oblique MRR	oblique recall@5	auto hit@1
BM25 only	70.0%	0.838	0.063	12.5%	97.3%
dense only (max-norm)	80.0%	0.887	0.192	31.3%	95.0%
fused lex 1 / dense 1	**90.0%**	**0.942**	**0.177**	**25.0%**	**97.3%**
fused lex 1 / dense 2 (shipped default)	**90.0%**	**0.942**	**0.181**	**12.5%**	**96.6%**
fused lex 1 / dense 2, depth 60	90.0%	0.942	0.167	12.5%	96.6%
fused lex 1 / dense 3	85.0%	0.917	0.181	12.5%	96.4%
fused lex 0.5 / dense 3	80.0%	0.900	0.186	25.0%	96.2%

Table 4. Lexical/dense balance (`scripts/tune-fusion.mjs`). "lex a / dense b" are the RRF list weights; depth is the number of candidates taken from each list (30 unless stated).

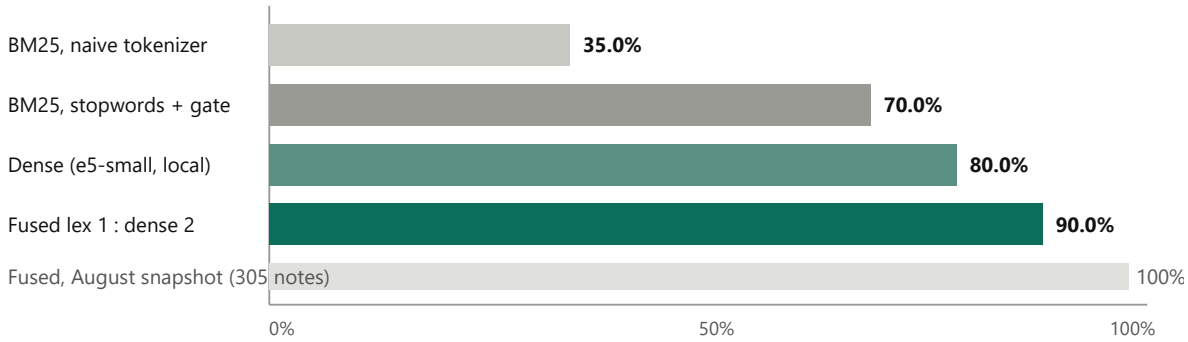

Figure 1. Curated hit@1 by ranker on the 479-note vault, with the August measurement of the same fused configuration on the 305-note snapshot for comparison.

> **Growth costs precision.** The same fused configuration measured 100% curated hit@1 and 1.000 MRR on the vault's 305-note snapshot of 9 August 2026, and 90.0% / 0.942 on the 479-note vault of 4 September. Two of twenty hand-written queries now rank their note second or third behind a newer note on the same subject. Any growing memory behaves this way, and it is the reason the regression gate exists: the drift was found by re-running the harness before any user noticed it.

6.5 Chunking made it worse, and that is informative

Splitting notes into passages before embedding is the standard next step in retrieval-augmented generation, and the obvious one here: a whole note averaged into one vector loses the paragraph that answers, and anything past the truncation point is invisible. We implemented

contextual chunking [8], each passage prefixed with the note's title and description, with three aggregations from passage scores to a note score, and swept passage size (Table 5).

passages	configuration	aggregation	curated hit@1	curated MRR	oblique MRR	auto hit@1
479	**single, header + first 900 chars**	**max**	**95.0%**	**0.967**	**0.210**	**97.5%**
1,150	chunks of 1,400, first 3 per note	mean2	95.0%	0.975	0.182	96.6%
479	single, header + full body	max	95.0%	0.975	0.115	96.2%
1,150	chunks of 1,400, first 3 per note	max-norm	90.0%	0.950	0.181	96.2%
3,742	chunks of 1,000, every passage	max-norm	90.0%	0.942	0.252	92.8%
479	single, header + first 1,400 chars (shipped default)	max	90.0%	0.942	0.181	96.6%
3,742	chunks of 1,000, every passage	max	90.0%	0.942	0.038	96.6%
1,150	chunks of 1,400, first 3 per note	max	85.0%	0.917	0.141	96.6%
3,742	chunks of 1,000, every passage	mean2	80.0%	0.883	0.049	96.8%

Table 5. Chunking sweep (`scripts/tune-chunking.mjs`) with the fused ranker. "single" embeds one passage per note (title + description + the first *n* characters of the body); "chunks" embeds every passage. Aggregations: *max* (best passage), *max-norm* (best passage damped by the logarithm of the passage count), *mean2* (mean of the two best).

The sweep was run twice, on the 305-note snapshot in August and on the 479-note vault in September, and the two readings differ in an instructive way. In August, splitting every note into passages cost 10 to 15 points of curated hit@1 outright. In September (Table 5) the cost moved: with plain max-pooling, chunking every passage ties the single-passage default on the curated set (90.0%) but collapses the oblique set (MRR from 0.181 to 0.038). The mechanism is the same in both cases. With roughly eight times as many passages as notes, max-pooling gives a long note one chance per passage to match by luck, so long retrospectives and legal texts float up. That is the length bias BM25 normalizes away and dense retrieval over whole documents does not; where the query shares no vocabulary with the note, chance is the only remaining signal. Four findings follow.

First, **plain max over every passage is never the best configuration**, on either vault. Second, **a length-aware aggregation repairs what max breaks, at a price**: damping the best passage by the logarithm of the passage count gives the best oblique MRR of any configuration (0.252) and costs four points on the auto set, the price of 3,742 passages competing for every query. Third, **bounded chunking is a middle ground**: the first three passages of each note with the mean of the two best scores reaches the highest curated MRR (0.975) at no auto cost. Fourth, **passage length matters as much as passage count**: a single passage of the header plus the first 900 characters beats the shipped 1,400-character default on every set (95.0% / 0.967 / 0.210 / 97.5%), while the full body ties on the curated set

and loses the oblique one: on notes that state their fact at the top, the tail is elaboration that blurs the vector. The differences among the top three configurations are one curated query and two oblique queries, within noise for this golden set; what the sweep establishes firmly is the bottom of the table. The shipped default stays at one 1,400-character passage until the gap register (§7) supplies enough real oblique queries to decide; the passage cap and size are explicit parameters, and vaults of long, multi-topic documents, where the answer can sit in the middle of a note, should expect the balance to tip toward chunking.

One result from the August sweep, not repeated here, should be stated separately: **the contextual prefix is not optional.** A passage stripped of its note's title and description loses its subject, and every configuration collapsed without it (curated hit@1 between 55% and 75%, auto between 73% and 79%, against 95 to 100% and 96% with it). Whatever the chunking, a passage must carry its document's subject.

6.6 Status demotion

Since v0.2 every ranker is wrapped so that notes with `status` *quarantine* or *deprecated* keep half their score and *archived* notes a quarter, looking past the requested depth so a demoted note that fell out of the top can be replaced. On a vault with no such notes the wrapper is a no-op, and the regression gate confirms no metric moved. On a fixture with an active note and a deprecated twin of identical wording, the twin ranks second with its status reported on the hit. The multiplier is deliberate rather than a hard filter: a deprecated note still surfaces when nothing better exists, and a person can see why it ranked where it did.

7. The gap register: learning from questions without storing them

A memory that several agents read on behalf of customers and colleagues faces a question the retrieval numbers do not answer: how does it learn from the people asking, without putting their conversations into a vault that lives in git forever? The answer is that it does not need their text. It needs to know **which questions it could not answer**: one row per question, not a transcript, which is actionable, small, and free of personal data by construction rather than by discipline.

The register records every `brain_search`, redacted, into a SQLite file outside the vault. Two signals carry the weight. **followed**: a search after which the same identity opens one of its results within ten minutes helped; a search nobody reads did not. The server sees this alone; since the 2026-07-28 revision there is no session in which "the same caller later read a result" could be expressed, so the server keeps its own short memory per agent and treats a read whose name appears in a recent result list as following that search. **count**: rows group by meaning, through the same local embedding model the ranker uses, so paraphrases collapse into one line; by normalized content words until the model has warmed up. The caller recorded is the agent, never a person.

The grouping threshold was measured rather than guessed. On Italian restaurant and operations questions, query-prefixed on both sides, five paraphrase pairs scored between 0.908 and 0.943 cosine, four unrelated pairs between 0.752 and 0.835, and "same topic, different question" pairs ("opening hours" vs. "are you open for lunch?") between 0.865 and 0.921. The default threshold, 0.9, sits in the gap between the highest unrelated and the lowest paraphrase score; it is a parameter, and the register should read as one row per question.

The register is a queue, not a memory: its job is to be emptied. A gap is a golden-set entry missing its answer. When a person writes the note that answers it and closes the gap naming that note, the pair (the question as the asker phrased it, the note the curator wrote) becomes an *oblique* query by construction, the kind that §5 shows the ranker is weakest on and that until now had to be written by hand imagining how someone else would ask. The regression gate then keeps every closed gap reachable when the ranker changes. What the register cannot see is "there, but wrong": a confident wrong answer arrives with a high score and a read and looks like a success. Only the agent or the person can say so; a `brain_feedback` tool files that verdict next to the question it came from.

8. Limitations and threats to validity

- **One corpus.** Every number is from one private vault in one language, written by one person. The findings on tokenization, graph expansion and chunking are stated as findings about *atomic, front-loaded notes*; on long, multi-topic documents the chunking result in particular is expected to reverse, and we say so.
- **A small, author-written golden set.** Twenty curated and eight oblique queries were written by the vault's author, who also wrote the notes; a single query is worth five and 12.5 points of hit@1 respectively, and no significance testing is meaningful at that size. The auto set is large but synthetic. The gap register exists partly to replace author-written oblique queries with real ones.
- **No user study, no end-task metric.** We measure whether the right note is retrieved, not whether an agent then answers correctly.
- **One embedding model, one hardware setting.** The dense results are for a small quantized multilingual encoder on CPU, chosen for privacy and cost; larger models may change the lexical/dense balance.
- **Snapshots.** The August and September measurements are on different vault sizes, which is the point of §6.4's drift finding but also means the two tables are not a controlled comparison.
- **What the next version needs.** Two or three corpora of different shape (long multi-topic documents, technical documentation dense with identifiers, real support questions), so that the chunking result can become a conditional rule rather than an observation; oblique queries collected by the gap register instead of written by the author; an end-to-end measure (answer correctness, groundedness, source attribution, token cost and latency) alongside retrieval; and the same golden set run against an extraction-based memory system on the same corpus.
- **Security claims are architectural, not audited.** The visibility-before-indexing property is enforced by construction and covered by tests; the PII and injection scanners are regular expressions, meant to fail closed at the gate and to be noisy but useful in lint, not to be complete.

9. Conclusion and availability

On the vault studied, Manent shows that agent memory can be plain files under version control without giving up retrieval quality, access control or auditability; what made that possible was measuring the retrieval at every change, which caught one regression the users had not noticed. For memories made of atomic, front-loaded notes, and pending replication on other corpora,

the measurements suggest four rules: fix the tokenizer before adding a model; fuse lexical and dense retrieval by rank and re-sweep the balance as the vault grows; never let every passage of a long note compete under plain max-pooling; and never embed a passage without its document's subject. The serving model says something about shared memories: decide who may read a note before you index it, keep agents' writes apart until a person promotes them, and learn from unanswered questions instead of storing conversations. The code, the specification, the golden set, the sweep scripts and the raw outputs behind every table are at github.com/colopisalvatore/manent under the Apache-2.0 license.

References

1. C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.
2. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST 2023; arXiv:2304.03442.
3. P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.
4. P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025.
5. S. Robertson, H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval 3(4), 2009.
6. L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei. *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672, 2024.
7. G. V. Cormack, C. L. A. Clarke, S. Buettcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.
8. Anthropic. *Introducing Contextual Retrieval*. anthropic.com/news/contextual-retrieval, September 2024.
9. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
10. T. H. Haveliwala. *Topic-Sensitive PageRank*. WWW 2002.
11. S. Ahrens. *How to Take Smart Notes*. 2017.
12. Model Context Protocol. *Specification, revision 2026-07-28*. modelcontextprotocol.io/specification/2026-07-28.

Reproduction: `paper/README.md` in the repository lists the exact commands; `paper/results/` holds the raw outputs the tables were transcribed from. The reference vault is private; the harness runs unchanged on any vault that conforms to the specification.